



initAEKF: Initializing the AEKF

- In this lesson, you will learn how to implement the AEKF in Octave, and will apply it to the nonlinear spring-mass-damper.
- The AEKF initialization code is similar to its EKF counterpart.

```
function aekfData = initAEKF(xhat0, SigmaX0, priorU, model)
    aekfData.xhat = xhat0;           % Initial state description
    aekfData.SigmaX = SigmaX0;       % Initial estimation-error covariance
    aekfData.SigmaV = model.SigmaV; % (Initial) sensor-noise covariance
    aekfData.SigmaW = model.SigmaW; % (Initial) process-noise covariance
    aekfData.priorU = priorU;        % Previous value of input force
    aekfData.model = model;          % Store model data structure too

    % For adaptive code: Storage of innovations and residues
    NV = 1000; % Length of buffer of past SigmaV to average
    aekfData.Vstore = zeros(length(model.SigmaV(:)), NV);
    NW = 1000; % Length of buffer of past SigmaW to average
    aekfData.Wstore = zeros(length(model.SigmaW(:)), NW);

    aekfData.iter = 0; % the present iteration number "k"
end
```



iterAEKF: Unpacking parameters

- The iterAEKF function is very similar to the iterEKF function, but we will still look at all of it.
- The function begins by unpacking parameters from aekfData.

```
function [xhat, bounds, SigmaW, SigmaV, aekfData] = iterAEKF(uk, zk, dT, aekfData)
    model = aekfData.model;

    % Get data stored in ekfData structure
    priorU = aekfData.priorU;
    SigmaX = aekfData.SigmaX;
    SigmaV = aekfData.SigmaV;
    SigmaW = aekfData.SigmaW;
    xhat = aekfData.xhat;
```



iterAEKF: Steps 1a–1c

- The Octave code next implements Steps 1a–1c.

```
% EKF Step 0: Compute Ahat[k-1]
Ahat = [1 dT; -model.k1*dT/model.m-3*model.k2*dT*xhat(1)^2/model.m ...
        1-model.b*dT/model.m];

% Step 1a: State prediction time update
xhat = [1 dT; -model.k1*dT/model.m 1-model.b*dT/model.m]*xhat + ...
        [0; -model.k2*dT*xhat(1)^3/model.m] + [0; dT/model.m]*priorU;

% Step 1b: Prediction-error covariance time update
%      sigmaminus(k) = Ahat(k-1)*sigmaplus(k-1)*Ahat(k-1)' + ...
%                  Bhat(k-1)*sigmawtilde*Bhat(k-1)'
% Note that Bhat is taken into account already in SigmaW by initAEKF.m
SigmaX = Ahat*SigmaX*Ahat' + SigmaW;

% Step 1c: Output prediction
zhat = atan(xhat(1)/model.d);
```



iterAEKF: Steps 2a–2c

- The Octave code next implements Steps 2a–2c.

```
% Step 2a: Estimator gain matrix
Chat = [model.d/(model.d^2 + xhat(1)^2) 0]; Dhat = 1;
SigmaZ = Chat*SigmaX*Chat' + Dhat*SigmaV*Dhat';
L = SigmaX*Chat'/SigmaZ;

% Step 2b: State estimate measurement update
mu = zk - zhat; % innovation. Use to check for sensor errors...
xhat = xhat + L*mu;
zhatplus = atan(xhat(1)/model.d);
r = zk - zhatplus; % residue

% Step 2c: Error covariance measurement update
SigmaX = SigmaX - L*SigmaZ*L';

[~,S,V] = svd(SigmaX); % Implement Higham method to help robustness
HH = V*S*V';
SigmaX = (SigmaX + SigmaX' + HH + HH')/4;
```



iterAEKF: Adapting $\Sigma_{\tilde{w},k}$ and $\Sigma_{\tilde{v},k}$

- The Octave code next computes covariance estimates.

```
% For adapting SigmaW
alpha = 0.99; % simple filter pole location
newW = (L*mu)*(L*mu)';
aekfData.Wstore = [aekfData.Wstore(:,2:end), newW(:)]; % Buffer result
if aekfData.iter > size(aekfData.Wstore,2) % Smooth with buffer AND filter
    newW = sum(aekfData.Wstore,2); % WARNING: Should be mean and not sum
    aekfData.SigmaW(:) = alpha*aekfData.SigmaW(:) + (1-alpha)*newW;
end
% For adapting SigmaV
newV = r*r' + Chat*SigmaX*Chat';
aekfData.Vstore = [aekfData.Vstore(2:end), newV(:)]; % Buffer result
if aekfData.iter > size(aekfData.Vstore,2) % Smooth with buffer AND filter
    newV = mean(aekfData.Vstore,2);
    aekfData.SigmaV(:) = alpha*aekfData.SigmaV(:) + (1-alpha)*newV;
end
```

- $\Sigma_{\tilde{w}}$ found using the “correct” equations is far too small. I use “sum” instead of “mean” to get a larger (proportional) result that seems to work well.



iterAEKF: Cleanup

- Finally, the code saves data for the next iteration.

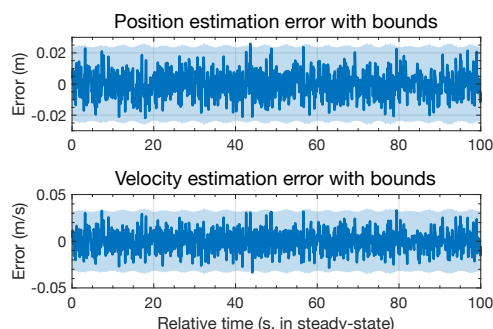
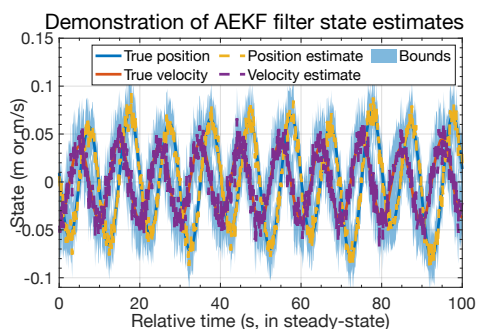
```
% Save data in aekfData structure for next time...
aekfData.priorU = uk;
aekfData.SigmaX = SigmaX;
aekfData.xhat = xhat;
bounds = 3*sqrt(diag(SigmaX));

aekfData.iter = aekfData.iter + 1; % Increment present iteration "k"
SigmaW = diag(aekfData.SigmaW); % Output to calling function
SigmaV = aekfData.SigmaV; % Output to calling function
end
```



Example AEKF results

- Some sample AEKF results are shown below, showing estimates plus bounds and errors plus bounds once the AEKF had reached steady-state.
- Results are fairly noisy, but confidence bounds are reliable.



Summary

- You have learned how to implement the AEKF in Octave code.
- Very similar to generic EKF but with code to adapt $\Sigma_{\tilde{w}}$ and $\Sigma_{\tilde{v}}$.
- $\Sigma_{\tilde{v}}$ converges to a value very close to truth (which is 10^{-3}).
- $\Sigma_{\tilde{w}}$ converges, but it is difficult to know “truth” since w_k enters the state equation in a nonlinear way.
- In this example, AEKF produces estimates that might be a good starting point for additional manual tuning.
- In other cases, the AEKF estimates may be good to use without further refinement.

